

Comparative Analysis of Time-Frequency Representations for EEG-Based Parkinson's Disease Diagnosis

Bezaleel Olakunori
Faculty of Science and Environment
MSc Data Science
Northumbria University
Newcastle upon Tyne, United Kingdom
bezaleel.olakunori@northumbria.ac.uk

ABSTRACT- Clinical diagnosis of Parkinson's disease relies on the judgement of movement disorder specialists and carries a documented misdiagnosis rate, with pooled accuracy estimated at about 80% [4,5]. Resting state EEG test is a non-invasive and low-cost medical diagnosis test that produces signals carrying oscillatory markers of the brain's signals [3], which makes it a candidate marker for diagnosing the brain activity for Parkinson's disease differentiators.

The aim of the study is to compare three Continuous Wavelet Transform (CWT) methods (i.e. the Gaussian 2nd derivative (gaus2), the Complex Gaussian 7th derivative (cgau7), and the Morlet (morl)) as time-frequency representations of resting state EEG for Parkinson's disease classification. One fixed CNN architecture was designed for a base comparison. Unique instance of this base model was trained on data sourced using all three methods so that performance differences can be attributed to the associated methods.

The Gaussian 2nd derivative time-frequency representation was the best performing model with an accuracy of 83.2% although it had a cross-validation accuracy of 76.58% and an AUC value of 0.898. The worst performing instance of the base model was the one that was trained using the Complex Gaussian 7th derivative time-frequency representation, which had an accuracy of 67.7% and an AUC value of 0.760. Training curves showed that the models were still improving when the fixed 15-epoch budget ended, so their figures are lower bounds for the architecture. The choice of time-frequency representation method affected the metric values of the models.

Keywords- parkinsons disease, eeg, cwt, wavelet transform, scalogram, deep learning.

1. INTRODUCTION

Parkinson's disease is a neurodegenerative, irreversible, and complex brain disorder affecting the dopaminergic neurons in the substantia nigra and is characterized by rigidity, tremor, bradykinesia, and non-motor symptoms such as sleep disorders, fatigue, dementia, bowel dysfunction, and other neuropsychiatric symptoms [1,2]. Parkinson's disease is the second most prevalent neurodegenerative disorder second only to Alzheimer's disease [1]. Despite years of research into this disease, there exists no cure yet for Parkinson's disease and it is difficult to diagnose even by experienced physicians [2]. The progression of Parkinson's disease begins before medical diagnosticians can clinically detect it, which makes early identification and prevention of the symptoms of vital importance [1].

Currently, Parkinson's disease diagnosis remains largely dependent on clinical observations of the patient's motor symptoms and their response to Parkinson's disease management medications [3]. This clinical approach comes with its limitations. One of the limitations is that motor symptoms are known to manifest only after a significant proportion of dopaminergic neurons reportedly within the range of 60% to 80% of the dopaminergic neurons in the substantia nigra would have already been lost [3]. This implies that by the time a clinical diagnosis is made, the patient would have experienced significant and maybe

irreversible neuron damage to their brain. In addition to that, the overlapping nature of Parkinson's disease symptoms with parkinsonian-syndromes such as multiple system atrophy and progressive supranuclear palsy contributes to the uncertainty of diagnosis even among medical specialists [4]. Postmortem confirmation studies have reported positive predictive values for idiopathic Parkinson's disease as low as 85.3% in specialist settings [4], while diagnosis done by movement experts rose from a diagnostic accuracy of about 79.6% to only 83.9% after clinical follow-up and refinement [5]. The limitations of the conventional clinical diagnosis methods have motivated researchers to develop objective, biomarker-driven diagnostic techniques that can detect neurophysiological brain changes earlier and more reliably than conventional clinical observation methods [3].

Among all the candidate markers used for the clinical diagnosis of Parkinson's disease symptoms, Electroencephalography has been the most promising candidate [3]. Contrary to other neuroimaging techniques that demanded expensive medical equipment and specialized medical facilities, EEG is a relatively low-cost and non-invasive technique [3]. EEG has shown impressive accuracies of 62% to 99.62% when used in advanced computational techniques such as the classification of Parkinson's disease using machine learning [3].

Despite the proliferation of these techniques, the existing studies lack a systematic, comparative evaluation of how different time-frequency representations perform specifically in the context of EEG-based Parkinson's disease diagnosis. Many studies adopt a single time-frequency method, design one or more models, optimize a model around it, and evaluate the performance of the model without comparing against alternative CWT methods using consistent conditions [6,7]. This has made it difficult to judge which time-frequency method offers better performance for Parkinson's disease diagnosis with EEG. The choice of CWT method is rarely justified despite the distinct time-frequency representation they exhibit. This also raises questions about whether the Parkinson's disease classification statistics of these models are based on true classification power or artifacts of the pipeline, dataset or feature extraction strategy.

Ethics and Limits of Use

The dataset used in this study is sourced from a publicly available secondary dataset published by Alex Rockhill on behalf of UC San Diego in California USA [8]. All the participant details had been removed adhering to data protection regulations. Ethical approval was granted under Northumbria University's research governance policies. All the data processing was done on a computer with documented and reproducible steps. No attempts were made to republish the dataset or any of its modified form.

2. LITERATURE REVIEW

Parkinson's disease is the second most common neurodegenerative disorder after Alzheimer's disease [9]. Parkinson's disease is also the third most significant health threat to middle-aged and older adults right behind cancer and cardiovascular diseases [9]. The symptoms of Parkinson's disease accumulate silently over time, and by the time the person is undergoing clinical trials, the case would have matured [9]. The Global Burden Disease dataset placed the worldwide prevalence of Parkinson's disease at approximately 11.7 million cases in 2021 [9,10]. This figure has gone up from 2.5 million cases in 1990, reflecting that the cases have more than tripled in a little over 3 decades [9,10]. In the U.S., the total economic burden of Parkinson's disease reached \$82.2 billion in 2024 [10].

2.1 Parkinson's Disease in Adults

In several Parkinson's disease studies, the prevalence of Parkinson's disease has been linked to aging, population growth, and epidemiological changes [9]. Aging was reported to contribute to 24.98% in males and 23.17% in females [9]. In other reports by the Parkinson's Foundation, the young (i.e. before age 50) onset of Parkinson's disease accounted for only 5-10% of all diagnosed case [10]. The global demographic shift towards an aging population has been anticipated to lead to a surge of a 1.5-fold increase in Parkinson's disease diagnosis by 2035 [9].

2.2 Machine Learning in Clinical Prediction

Machine learning [10]. Convolutional Neural Network (CNN) is a class of deep learning architectures designed for grid-structured data [12,13]. CNN models can learn features about the input data without needing to be given features from which to learn from [12]. This attribute of CNNs has made it popular in medical image analysis over the years, with many researchers choosing to use CNN models in organ segmentation, tumour detection, disease classification, and Parkinson's disease diagnosis [14].

In clinical prediction, CNNs have been applied to a variety of data forms such as brain Magnetic Resolution Images (MRI) for neurological diagnosis, X-ray imaging detecting infectious diseases such as COVID-19, and physiological signals such as EEG and ECG signals [15].

2.3 EEG Signals

An Electroencephalography (EEG) test is a painless and risk-free medical diagnosis procedure that is used to monitor and analyze electrical pulses in the brain. An EEG test is able to be used to identify issues with a person's brain activity based on the fact that the brain cells communicate with the use of electrical activity. With the use of electrodes that are wire-strapped to the scalp at specific locations, EEG devices (as shown in Figure 1) can capture brain wave patterns by recording the electrical impulses and transmitting them to a computer. These waves (shown in Figure 3) can then be shown on a computer and analyzed by the diagnostician. The EEG signals used in the external dataset utilized in this study were gotten from a BioSemi device, which means the layout standard of the electrodes (shown in Figure 2) is the biosemi32 standard.

Figure 1: Real-life EEG recording setup



Figure 2: Sensor locations plotted in Python with the biosemi32 standard

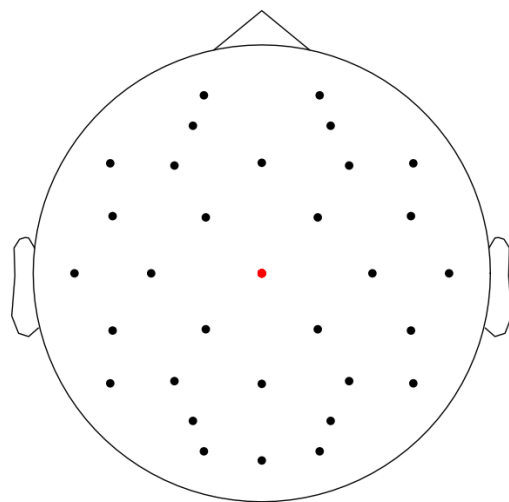
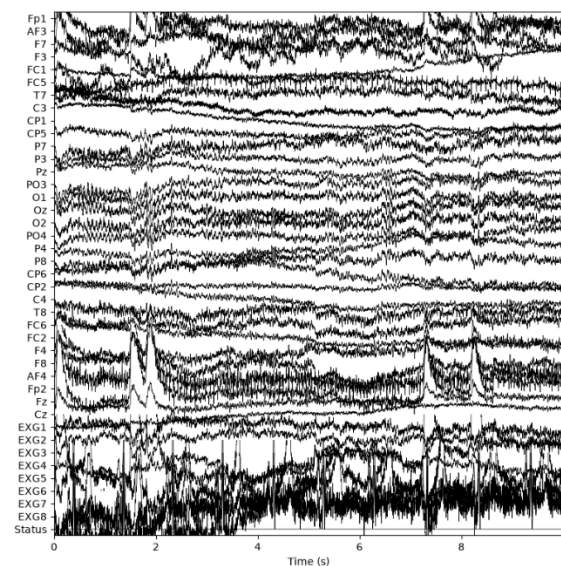


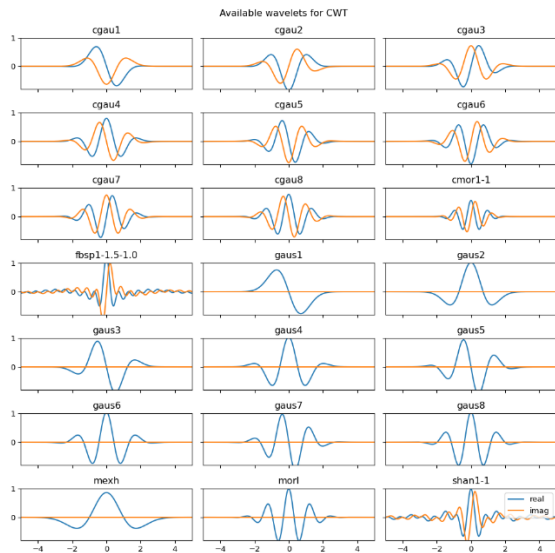
Figure 3: Raw EEG signal for a healthy participant



2.4 Continuous Wavelet Transform

Wavelet analysis is a popular tool for analyzing localized variations of power within a time series such as EEG signals [16]. The Continuous Wavelet Transform is a 2D representation of time and frequency data [16]. Several variations of the CWT exist with each one having distinct properties. Some people have mentioned that the choice of the wavelet function is arbitrary when in fact there are specific factors to consider when choosing a wavelet function [16]. Complex wavelet functions are better suited to capturing oscillatory behaviour due to their ability to retain information about the amplitude and phase [16].

Figure 4: Graphs of CWT wavelet methods available in PyWavelet



2.5 Scalogram

Scalogram is the time-frequency representation obtained through a CWT, which is defined as $|W_x(a, b)|^2$, and computed across a range of scales a and time shifts b [17,18]. Since biomedical signals such as EEG signals contain both slow oscillatory rhythms, and brief transient events that a fixed-window spectrogram would tend to misrepresent, the scalogram is well-suited to represent EEG signals.

The type of wavelet affects the resulting scalogram and by extension, the Parkinson's disease classification performance. Wavelet families such as the Mexican hat, Symlet, and Daubechies have been explored, but comparative studies rarely compare the different wavelet types [18,20].

Scalograms are generated as 2D images, the pixels and channels are extracted into array-like structures, and fed to CNN models, which would then proceed to learn the discriminating time-frequency patterns [17]. This has been applied to various studies such as schizophrenia detection from EEG, ECG arrhythmia detection, motor image classification from EEG, and Parkinson's disease diagnosis from EEG [6,17,18,19]. In several studies, scalograms derived from CWT have been shown to consistently outperform discrete wavelet transforms [6]. The scalograms of a subject diagnosed with Parkinson's disease albeit with different time-frequency representations is shown in Figure 5, Figure 6, and Figure 7.

Figure 5: Scalogram of a Parkinson's disease subject using the Gaussian 2nd derivative wavelet method

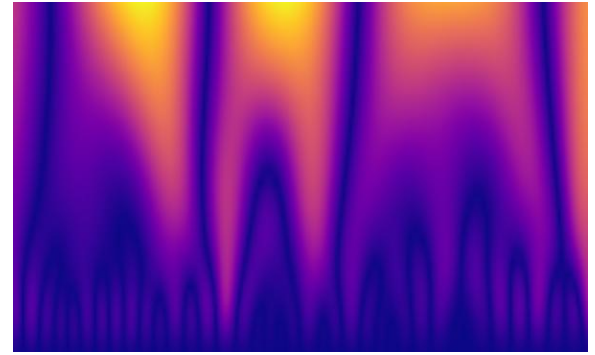


Figure 6: Scalogram of a Parkinson's disease subject using the Morlet wavelet method

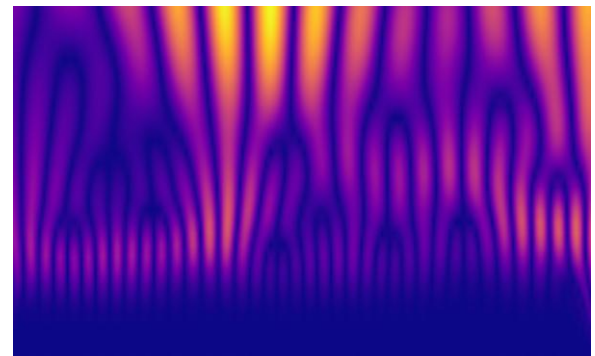
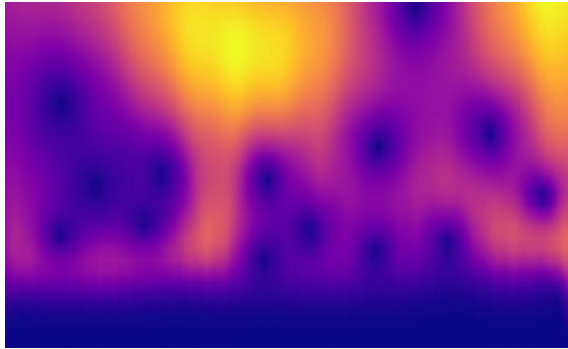


Figure 7: Scalogram of a Parkinson's disease subject using the Complex Gaussian 7th derivative wavelet method



2.6 Research Gap and Significance of the Study

Various time-frequency methods have been devised and applied to study Parkinson's disease diagnosis using resting state EEG signals. Each of these methods have specific theoretical foundations, compute requirements, and trade-offs between the resolution of time and frequency. The Short-Time Fourier Transform (STFT), which is used for the extraction of narrow-band frequency in noisy signals is still widely used due to its simplicity and ease of interpretation [21]. However, the fixed-length and small window it uses imposes an unavoidable trade-off between temporal and spectral precision [21]. The CWT is an alternative type of time-frequency representation that builds on multiresolution signal decomposition framework of Mallat to provide finer temporal resolution at high frequencies and finer spectral resolution at low frequencies [21,22]. This addresses the limitation of the STFT method through multiresolution analysis, making it more suitable for capturing transient neural events [21,22].

Modern techniques such as the Hilbert-Huang Transform (HHT), the Wigner-Viller Distribution (WVD), and Variational Mode Decomposition (VMD) have tried to overcome the constraints of linear methods by adaptively decomposing the signal or pursuing time-frequency localization at the expense of linearity, which has led to increased compute complexity, and in some cases cross-term interference and mode mixing [23,24].

Despite the spread of these techniques, the existing studies lack a systematic, comparative evaluation of how different time-frequency representations of

model data sourced from EEG signals impact the Parkinson's disease diagnosis model. Many existing studies utilize a single CWT method, develop a model around it, and evaluate the performance of the model without considering alternative CWT methods. This has made it difficult to make firm conclusions about which CWT time-frequency method offers the most discriminative capabilities for diagnosing Parkinson's disease subjects from healthy subjects.

This study contributes to the growing effort to develop an objective, non-invasive and timely model for the classification of Parkinson's disease from resting state EEG signals. By developing and evaluating multiple time-frequency representations based on varying CWT methods, this study looks to provide clarity on the effect of different CWT methods on the performance of Parkinson's disease classification from resting state EEG signals. The findings from this study are intended to offer insight into the CWT method that yields a better performance in Parkinson's disease classification with the aid of machine learning models.

2.7 Research Aim

The aim of this dissertation is to compare the performance of three different time-frequency representations of EEG signals in the classification of Parkinson's disease using a publicly available dataset. The classification model would be developed after applying various data cleaning and scaling techniques to deal with noisy EEG signals and low volume of medical dataset. The study would also employ a variety of evaluation metrics to quantitatively analyse the best time-frequency representations among the chosen set for classifying Parkinson's disease.

2.8 Research Objectives

1. Develop a pipeline to preprocess the dataset.
2. Perform exploratory data analysis on the dataset to identify noisy channels and trim unwanted artefacts.
3. Convert the EEG signals into scalograms using each of the time-frequency representation models.
4. Scale and split the dataset for each time-frequency representation.

5. Develop a base model for Parkinson's disease classification.
6. Assess the accuracy of the time-frequency methods using the base model.
7. Discuss the results of the findings for medical diagnosis.

2.9 Research Questions

This study aims to answer the following questions:

RQ1: How do different time-frequency representations affect the classification performance of EEG data used in Parkinson's disease diagnosis?

RQ2: What impact do time-frequency domain transformation have in Parkinson's disease diagnosis from EEG data?

3. METHODOLOGY

A comparative analysis of the performance of the same model that uses scalograms generated by three CWT methods will be carried out for a fair comparison.

3.1 Ethical Considerations

The dataset used for this study was sourced externally whilst adhering to their usage policies. The external dataset is publicly available and fully anonymised, which protects the identity of the participants. No further data was collected for the purpose of this study and the ethical approval guidelines of Northumbria University were adhered to.

3.2 Research Design

This research employed three major phases: data preparation, model development, and model evaluation. Python (version 3.12.6) was the main programming language used for all the data processing tasks. Its ease of use coupled with the convenience of storing computational results in Jupyter Notebooks made it easier to test the EEG data processing pipelines. Python also has a robust system of libraries that facilitate the execution of scientific research. Some of those libraries which were used in this research are: VirtualEnv to manage environments and the dependencies, MNE for reading the EEG signal files and performing computations on them, Autoreject to filter epochs, PyWavelets to convert MNE's epoch into

scalograms using CWT and any time-frequency representation method, Matplotlib for plotting graphics, NumPy for fast computation with array data structures, OpenCV for image reading, Tensorflow for model design, SciKit-Learn for splitting dataset and computing model evaluation metrics.

3.3 Dataset Description

The dataset I will use is a collection of resting state Electroencephalogram (EEG) signals that was collected at the University of San Diego and curated by Alex Rockhill [8]. The dataset was accessed on June 2026 and at the time of download, its license was Creative Commons Open (CC0), there were no data errors or warnings, and the latest version was 1.0.5 [8]. 31 participants were included in the data collection, and a total of 3 sessions were organized. Out of those 31 participants, 15 had Parkinson's disease and 16 were healthy [8]. The dataset was also gender-balanced given that there were 17 females and 14 males [8]. The age range of the subjects was 47 to 82 inclusive [8]. The sessions involved were signal collection of healthy participants and signal collection of participants with Parkinson's disease before and after taking some medication [8]. For this study, only the session before (i.e. the session called ses-off) taking the Parkinson's disease medications was considered.

The dataset contained a participants.tsv file, which was used to determine the folder containing the EEG data for each participant as well as other data such as the gender distribution [8].

3.4 Data Cleaning

EEG data can include noise and artefacts that could affect the outcome of analysis. Removing these elements can improve the outcome of analysis. The preparation of the EEG data for time-frequency transformation and the model training is also required. This study implements a pipeline for cleaning and preparing the EEG data using the mne, PyWavelets, Autoreject, OpenCV, and NumPy packages. Bad channels were removed by filtering out channels that weren't part of the EEG device's standard channels. To remove slow drifts and DC offset, the epochs were filtered to select frequencies within the low range of 1 and the high

range of 40, which were determined by manual inspections of the EEG plots.

3.5 Data Scaling

To increase the number of training and testing samples available for the study, the dataset was scaled using NumPy's "reshape" algorithm. This was done by reshaping each cleaned EEG epoch into a set of no more than 50 segments. For each session's multichannel epoch, the epochs were flattened and reshaped into a 2D matrix with a fixed column size of 512. The column size of 512 was chosen because it pronounced the differences of the time-frequency wavelet functions. The row size was left to be decided by NumPy based on the rows inferred from the length of the epochs array. Each row of the returned reshaped matrices set comprised of shorter epochs, allowing a large epoch to yield several smaller epochs. The number of reshaped epochs selected was capped at 50 due to high computational costs of processing larger epochs.

The reshaping of the epochs produced fixed-length segments with a uniform size, which we would have needed to do before generating the scalograms. This reshaping process employed a segmentation-driven data augmentation method rather than a synthetic signal generation data augmentation method. As such, no noises were introduced, there was no time-warping, and no generative data modelling was performed to expand the sample size. This led to the preservation of the characteristics of Parkinson's disease-relevant characteristics from the EEG signals despite the shorter fixed-length window and adheres to medical data ethics.

3.6 Feature Engineering and Encoding

3.6.1 Scalogram Generation and Normalization

For each cleaned and scaled epoch, the scalogram image was computed using each of the methods (i.e. Gaussian 2nd derivative, Complex Gaussian 7th derivative, and Morlet), a static y-axis scale from 1 to 33, and PyWavelet's cwt function. The converted image was then saved to the file system for persistence needs and to avoid duplicate computation of the time-consuming scalogram generation routine when the process crashes.

These pixels in each image were later scaled to the range of 0-1 by dividing the values by 255. This was needed to keep the values of the dataset in a common range and stabilize the training.

3.6.2 Encoding Status

The `to_categorical` method from Tensorflow's Keras utilities module was applied to the status column of each scalogram. The status could be healthy or unhealthy depending on whether a participant was a healthy control or diagnosed with Parkinson's disease.

3.6.3 Feature Selection (x) and Target (y)

The feature matrix x was sourced from the generated scalogram images and consisted of images with a dimension of 600 width, 365 height, and 3 colour channels. The target vector y consisted of only the outcome of the participant (0 or 1 representing healthy or unhealthy respectively).

3.7 Data Splitting

The scalogram images generated for each time-frequency method was split using an 80:20 ratio and the `train_test_split` method from SciKit-Learn with parameters `test_size=0.2`, and `random_state=42`. As a result, there were 1240 images in the training set and 310 images in the test set. The test set was not used for training purposes but rather prediction evaluation and validation of the models.

3.8 Model Development

To ensure a fair comparison of the various time-frequency representations, the same model architecture was used for each time-frequency representation dataset. The base model was a CNN model with 11 layers and 12,913,858 total and trainable parameters as seen in Figure 8.

Figure 8: Summary of the base model

Layer (type)	Output Shape	Param #
rescaling_5 (Rescaling)	(None, 365, 600, 3)	0
conv2d_15 (Conv2D)	(None, 363, 598, 32)	896
max_pooling2d_15 (MaxPooling2D)	(None, 181, 299, 32)	0
conv2d_16 (Conv2D)	(None, 179, 297, 64)	18,496
max_pooling2d_16 (MaxPooling2D)	(None, 89, 148, 64)	0
conv2d_17 (Conv2D)	(None, 87, 146, 64)	36,928
max_pooling2d_17 (MaxPooling2D)	(None, 43, 73, 64)	0
flatten_5 (Flatten)	(None, 200896)	0
dense_10 (Dense)	(None, 64)	12,857,408
dropout_5 (Dropout)	(None, 64)	0
dense_11 (Dense)	(None, 2)	130

3.9 Evaluation Metrics

The models for each CWT method are trained on their datasets (i.e. the model for a Morlet CWT is trained on Morlet scalogram images), and their prediction performance is computed using their test scalogram images and evaluated against the other methods. No special treatments were given to any model to ensure a fair comparison. Since we are only interested in the performance comparison of the methods, no attempt was made to tune the hyper parameters of each model. This was also to ensure a fair comparison and that a specific hyperparameter does not give one method an edge over another method.

The following metrics were collected for each model:

- Test accuracy: Measures the proportion of samples that were correctly classified by the model [23].
- Test loss: Measures how far the model's prediction deviates from the true labels on the test set [23]. Lower loss implies better fit of the model.
- Accuracy: The proportion of correct predictions (i.e. true positives and true negatives) out of all predictions made by the model [23].
- Precision: The proportion of true positive predictions and all the real positive predictions [23].
- Specificity: The proportion between true negative predictions and all negative predictions [23].
- Recall (sensitivity): The proportion of actual Parkinson's disease subjects that are correctly identified [23]. This helps to

identify the number of false negative classifications.

- F1-Score: The harmonic mean of precision and recall [23].
- ROC Curve: A graphical plot of true positive rates against false positive rates across all training epochs [24].
- Area Under the Curve (AUC): A summary metric of the Receiver Operating Curve (ROC), which represents the sensitivity versus the specificity with respect to a discrimination threshold and the model's discriminating ability [24].
- Cross-validation accuracy: A 5-fold cross-validation for the accuracy was carried out on the test data for each method using Sci-Kit Learn's KFold helper. This was used to provide a robust estimation of the accuracy of each method.

These metrics are enough to judge how each method's EEG time-frequency representation performs with respect to the constant base model.

4. FINDINGS AND DISCUSSIONS

4.1 EDA Findings and Visualizations

After cleaning the EEG signals, 50 reshaped epochs were generated from each of the 31 participants, so 1550 scalogram images were generated for each of the three methods. The noise from the EEG signals had been trimmed out, leaving a much cleaner graph behind. Figure 9 and Figure 10 show what the EEG signals for a subject look like before they were cleaned and after they were cleaned respectively. Figure 9, Figure 10, and Figure 11 belong to the same subject.

Figure 9: Uncleaned EEG signals of a Parkinson's disease subject

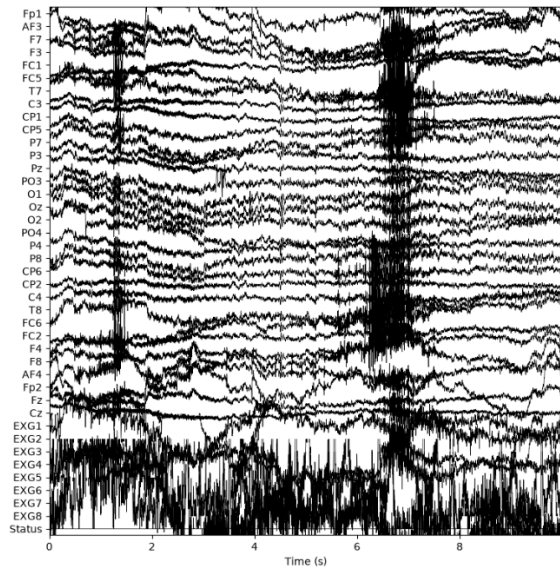


Figure 10: Uncleaned EEG signals of a Parkinson's disease subject

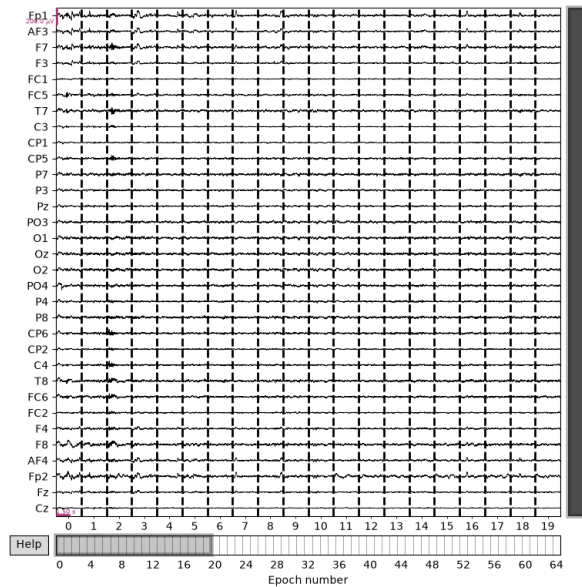
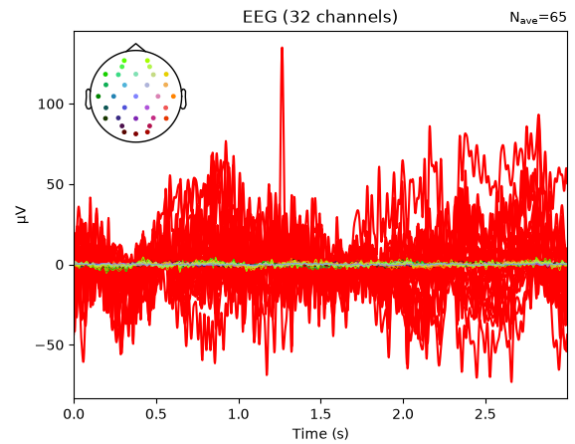


Figure 11: Plot of the rejected data (shown in red) from a Parkinson's disease subject



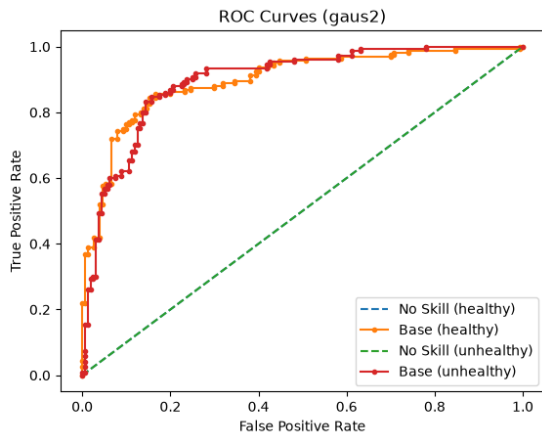
4.2 Gaussian Wavelet, 2nd Derivative Performance Results

An instance of the base model was trained and tested in isolation on the scalograms generated with the Gaussian wavelet, 2nd derivative (gaus2) method. Table 1 represents the classification report from the predictions of the model trained with gaus2 scalogram images. The confusion matrix for the model was $\begin{bmatrix} 137 & 23 \\ 29 & 121 \end{bmatrix}$, which shows that the model effectively classified most of the instances for both classes, with fewer false negatives indicating that there are fewer instances of Parkinson's disease going undetected.

Table 1: gaus2 classification report

	Precision	Recall	F1-score	Support
0	0.83	0.86	0.84	160
1	0.84	0.81	0.82	150
Accuracy			0.83	310
Macro Average	0.83	0.83	0.83	310
Weighted Average	0.83	0.83	0.83	310

Figure 12: ROC curves of the gaus2 model against the no-skill model



The gaus2 ROC curve (Figure 12) plotted the True Positive Rate against the False positive rate and generated an AUC of 0.898. According to [25], this value falls within the good to very good or high category.

Figure 13: Accuracy graph of the gaus2 model's training and validation

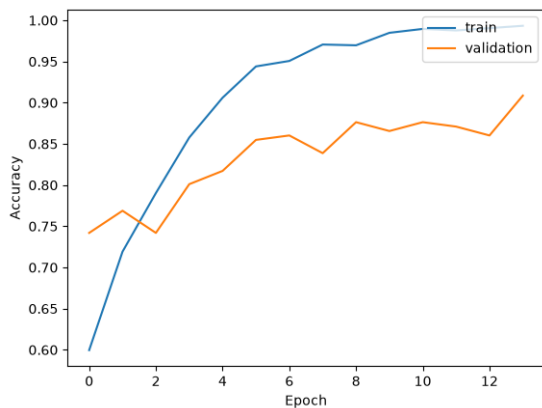
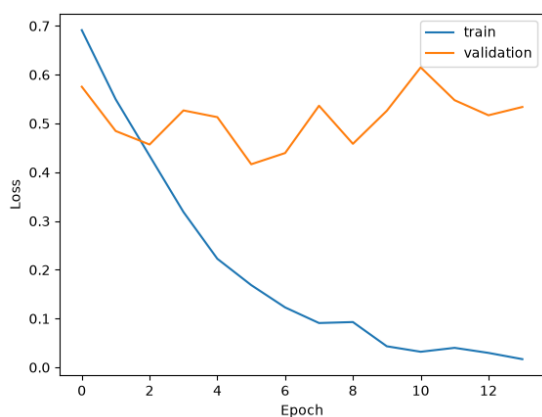


Figure 14: Loss graph of the gaus2 model's training and validation



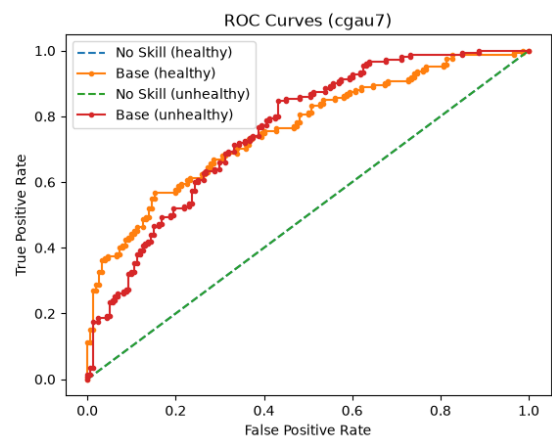
4.3 Complex Gaussian Wavelet, 7th Derivative Performance Results

An instance of the base model was trained and tested in isolation on the scalograms generated with the complex Gaussian wavelet, 7th derivative (cgau7) method. Table 2 represents the classification report from the predictions of the model trained with cgau7 scalogram images. The confusion matrix for the model was $\begin{bmatrix} 115 & 45 \\ 55 & 95 \end{bmatrix}$, which shows that the model effectively classified most of the instances for both classes, with fewer false negatives indicating that there are fewer instances of Parkinson's disease going undetected.

Table 2: cgau7 classification report

	Precision	Recall	F1-score	Support
0	0.68	0.72	0.70	160
1	0.68	0.63	0.66	150
Accuracy			0.68	310
Macro Average	0.68	0.68	0.68	310
Weighted Average	0.68	0.68	0.68	310

Figure 15: ROC curves of the cgau7 model against the no-skill model



The cgau7 ROC curve (Figure 13) generated an AUC of 0.760. According to [25], this value falls within the moderate to good category.

Figure 16: Accuracy graph of the cgau7 model's training and validation

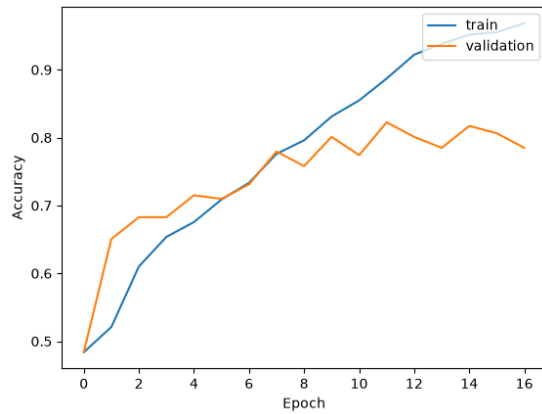
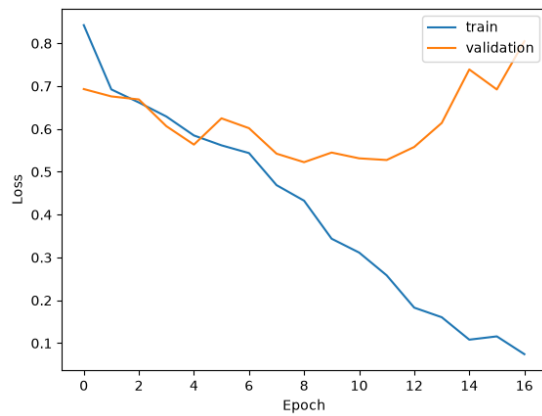


Figure 17: Loss graph of the cgau7 model's training and validation



4.4 Morlet Wavelet Performance Results

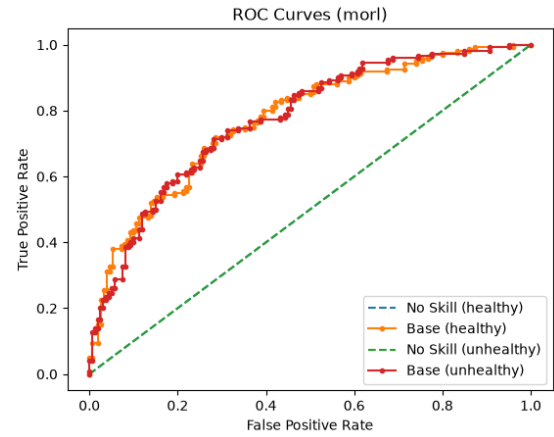
An instance of the base model was trained and tested in isolation on the scalograms generated with the Morlet (morl) method. Table 3 represents the classification report from the predictions of the model trained with morl scalogram images. The confusion matrix for the model was $\begin{bmatrix} 107 & 53 \\ 39 & 111 \end{bmatrix}$, which shows that the model effectively classified most of the instances for both classes, with fewer false negatives indicating that there are fewer instances of Parkinson's disease going undetected.

Table 3: morl classification report

	Precision	Recall	F1-score	Support
0	0.73	0.67	0.70	160
1	0.68	0.74	0.71	150
Accuracy			0.70	310

Macro Average	0.70	0.70	0.70	310
Weighted Average	0.71	0.70	0.70	310

Figure 18: ROC curves of the morl model against the no-skill model



The morl ROC curve (Figure 14) plotted the True Positive Rate against the False positive rate and generated an AUC of 0.775. This value falls within the moderate to good category [25].

Figure 19: Accuracy graph of the morl model's training and validation

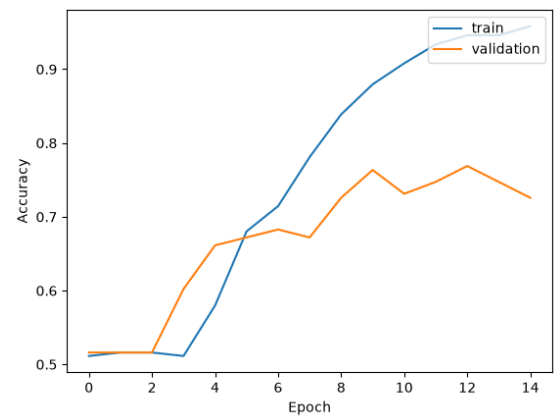
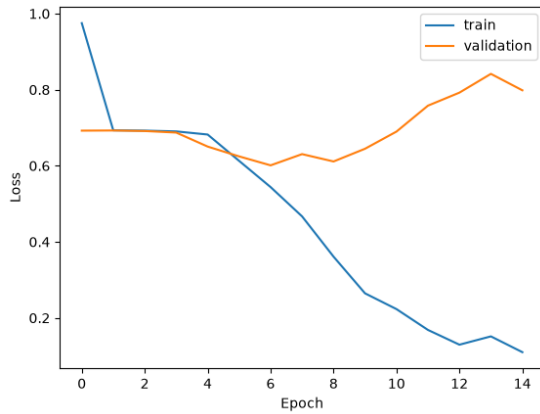


Figure 20: Loss graph of the morl model's training and validation



4.5 Cross-Validation Results

The 5-fold cross-validation (CV) was executed to obtain a more dependable estimate of each method's performance [26].

The CV scores were 76.58%, 58.71%, and 63.68% for the gaus2, cgau7, and morl wavelet methods respectively. The CV scores were less than the scores obtained from the test set's predictions, which indicates some overfitting to the training set in the models [26].

4.6 Wavelet Method Model Comparison

The Complex Gaussian 7th derivative wavelet method outperformed the Gaussian 2nd derivative and Morlet wavelet methods. A complete summary of the collected evaluation metrics is shown in Table 4.

Table 4: Summary of performance results

Method \ Metric	gaus2	cgau7	morl
Test Accuracy	0.832	0.677	0.703
Test Loss	0.580	0.618	0.570
CV Accuracy	76.58%	58.71%	63.68
CV Std Deviation	4.0%	6.78%	10.0%
AUC	0.898	0.760	0.775
Accuracy	0.832	0.677	0.703
Precision	0.840	0.679	0.677
Specificity	0.856	0.719	0.669
Recall	0.807	0.633	0.74
F1 Score	0.823	0.655	0.707

The gaus2 had the highest values for several evaluation metrics. Its recall was 0.807 with the next recall being 0.74 and belonging to morl, which means that it had fewer false negatives. Missing a classification of Parkinson's disease in a patient can be crucial, so the fact that gaus2 is less likely to incorrectly classify a patient with Parkinson's disease. The low recall reported by the cgau7 method indicates that the model is more likely to miss cases of Parkinson's disease.

The cgau7 method had the highest test loss, which indicates signs of overfitting. The morl method had the lowest test loss but didn't attain the highest test accuracy.

The method with the lowest specificity was morl, which indicates a higher chance of incorrectly classifying a healthy patient as unhealthy.

4.7 Alignment with Previous Studies

The results are not in alignment with previous studies. Whilst there are considerable performance differences in performance between each model, the accuracies obtained in this study is much less than the values obtained in previous studies. Previous studies managed to obtain accuracy figures of 97.28% [6], which is much higher than the 76.58% accuracy achieved by the best performing model in this study.

5. CONCLUSIONS

5.1 Critical Evaluation of Findings and Achievement of Research Aims

This study evaluated the performance of three time-frequency representations in a model for classifying Parkinson's disease using a research dataset of 31 participants aged 47 to 82 inclusive [8]. All the research objectives were carried out. The data was scaled from 31 unique sessions per participant to 1550 unique scalograms.

The accuracy of each time-frequency representation covered in this study whilst being moderate, answered the research aims. The Gaussian wavelet, 2nd derivative achieved an accuracy of 83.2% (AUC of 0.898, CV accuracy of 76.58%, and CV standard deviation of 4%), which outperformed the Morlet wavelet (accuracy of 70.3%, AUC of 0.775, CV accuracy of 63.68%, and CV standard deviation of 10%), and the

Complex Gaussian 7th derivative wavelet (accuracy of 67.7%, AUC of 0.677; CV accuracy of 58.71% with CV standard deviation of 6.78%). The varying results prove that different time-frequency representation methods can provide discriminating values that can affect the classification performance of models used in the research of classifying Parkinson's disease.

Another thing to note is that the test accuracies of the time-frequency wavelet methods (apart from the Morlet wavelet method) were outside 1 standard deviation from the CV accuracy.

5.2 Limitations

The limitations of the findings in this research must be acknowledged.

- The use of secondary data whilst being convenient, poses questions about the quality of the data and the sampling methods involved in the data collection.
- It is difficult to differentiate normal brain activity from activity affected by Parkinson's disease, which makes automated EEG cleaning difficult.
- The data scaling via reshaping means that the entire EEG signal is not trained on, and only a subsection is analyzed and classified as having the same status as the entire EEG session, which might not always be true.
- The training epoch was too short given that there was no convergence of the training curves.
- The running time of the training and evaluation process for each method was not tracked, which would have helped to advice on the cost incurred.
- Inability to test all the CWT methods exposed by PyWavelet and to train the models for longer epochs due to limited resources.
- The results can only be used for research and are not approved for clinical trials [8].

Regardless of the limitations mentioned above, the results from this study still help to address the aim of this study albeit within the constraints imposed by the limitations.

5.3 Recommendations

For future research we could test other time-frequency wavelet functions and analyze their performance. Alternative datasets with more subjects could be considered. During the scaling process, different scales were explored to arrive at the one that amplifies the differences of each wavelet method in the output scalograms, so this is something that could be explored in a future study. Although we performed a one-size fits all in this study to allow for a fair comparison between the different time-frequency representations, a further development of a more optimized model using the best performing method in this study might yield promising results that could advance our research into Parkinson's disease. It would also be useful to get access to more compute resources like a GPU to improve the training performance for all the models.

5.4 Concluding Remarks

The different models didn't show good performance during the study and performed poorer than the models developed in other studies. However, they addressed the research question and proved that different time-frequency wavelet representations do have different effects when used in the development of research models for the classification of Parkinson's disease from resting state EEG signals.

This research contributes to the existing literature by comparatively analysis three different time-frequency representations of EEG signals and showing how they impact the performance of a model designed for Parkinson's disease diagnosis.

APPENDIX A

JUPYTER NOTEBOOK OF RESEARCH

This appendix gives the codes that were executed in Jupyter Notebook to arrive at the results in this research.

Figure A.1: Imported modules

```
# import system modules
import gc
import os

# import 3rd-party modules
import cv2
import matplotlib.pyplot as plt
import mne
import numpy as np
import pandas as pd
import pywt as pywt

# import artifacts from 3rd-party modules
from autoreject import AutoReject
# from google.colab import drive
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, roc_curve
from sklearn.model_selection import KFold, StratifiedKFold, train_test_split
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.utils import to_categorical
```

Imports the system, scientific computing and deep learning modules used across the notebook.

Figure A.2: Dataset configuration

```
data_url = 'https://openneuro.org/datasets/ds002778/versions/1.0.5/download'

data_folder_root = ''
dataset_name = 'ds002778'

if "COLAB_RELEASE_TAG" in os.environ:
    from google.colab import drive

    drive.mount('/content/drive', force_remount=True)
    data_folder_root = '/content/drive/MyDrive/Colab Notebooks/data'
elif os.path.exists('/mnt/c/Users/bezal/'):
    data_folder_root = '/mnt/c/Users/bezal/Documents/Northumbria_University/KF7029/data'
else:
    data_folder_root = './data'

# column descriptions:
# participant_id: Unique participant identifier
# age: Age of the participant at time of testing. Units: years
# gender: Gender identity of the participant. Levels: F->female, M->male
# hand: Handedness of the participant. Levels: R->right, L->left, A->ambidextrous
# MMSE: Mini-Mental State Examination score.
# Levels:
# >24->normal cognition
# 19-23->mild cognitive impairment
# 10-18->moderate cognitive impairment
# <9->severe cognitive impairment
# NAART: North American Adult Reading Test score, scored similar to an IQ test on
# a 100-point-based scale,
# disease_duration: How long since the patient was diagnosed. Units: years
# r1_deficits: Which side of the body has deficits and how great these deficits are
# notes: Experimenter/curator notes on collection and curation
participants = pd.read_csv(f'{data_folder_root}/{dataset_name}/participants.tsv', sep='\t')
participants.describe(include='all')
```

Resolves the local data folder and loads the participants.tsv metadata table.

Figure A.3: Session construction

```
pd_participants = participants[participants.participant_id.str.startswith('sub pd')]
sessions = ['healthy', 'unhealthy']
session_type_to_status = {
    'sub hc': 'status hc',
    'sub pd': 'status pd',
    'sub off': 'status off'
}
sessions = pd.DataFrame()

for i in range(participants.shape[0]):
    session = participants.loc[i].copy()
    participant_id = participants.participant_id[i]
    session_type = 'sub hc' if participant_id.startswith('sub hc') else 'sub off'
    eeg_file_path = f'{data_folder_root}/{dataset_name}/{participant_id}/{session_type}/eeg/{participant_id}_{session_type}_task-rest_eeg.bdf'
    session['session_type'] = session_type
    session['eeg_path'] = eeg_file_path
    session['eeg_raw'] = mne.io.read_raw_bdf(eeg_file_path, preload=True, verbose=False)
    session['status'] = session_type_to_status[session_type]
    sessions = pd.concat([sessions, session])

print(sessions[participants_id, 'session_type', 'status'].tail(10))
print(f'Total number of participants: {participants.shape[0]}')
print(f'Number with Parkinson's disease: {pd.participants.shape[0]}')
print(f'Number without Parkinson's disease: {participants.shape[0]} - pd.participants.shape[0]')
print(f'Total number of sessions: {sessions.shape[0]}')
```

Builds the sessions dataframe, loading each participant's raw BDF recording and assigning the healthy or unhealthy status label.

Figure A.4: EEG file listing

```
# print the file path of all EEG BDF files in the sessions
i = 0
for session in sessions.iteruples():
    print(f'[{i}] {session.eeg_path}\n')
    i += 1
```

Prints the file path of every EEG BDF recording in the sessions dataframe.

Figure A.5: Raw signal inspection

```
for session in sessions.iteruples():
    print(f'Path: {session.eeg_path}')
    eeg_bdf = session.eeg_raw
    print(f'Data shape (Signals_Count, Time_Points_Count): {eeg_bdf.get_data().shape}\n')
    eeg_bdf.plot(
        title=f'EEG Signals for Participant {session.participant_id} ({session.session_type})',
        n_channels=41,
        show_scrollbars=False,
        show_scalebars=False)
```

Plots the unprocessed EEG signals of each participant for visual inspection.

Figure A.6: Bad channel marking

```
bad_channels = ['EXG1', 'EXG2', 'EXG3', 'EXG4', 'EXG5', 'EXG6', 'EXG7', 'EXG8', 'Status']
noisy_channels = ['F7', 'T7', 'C4', 'F8', 'Fp2', 'Fz']

for session in sessions.iteruples():
    # remove bad channels if they exist in the list of channels
    session.eeg_raw.info['bads'] = list(set(bad_channels).intersection(session.eeg_raw.info.ch_names))
```

Marks the non-EEG reference channels (EXG1 to EXG8, Status) as bad and lists the noisy channels observed during inspection.

Figure A.7: Montage assignment

```
for session in sessions.iteruples():
    montage = mne.channels.make_standard_montage('biosemi32')
    session.eeg_raw.set_montage(montage, on_missing='warn')
    session.eeg_raw.plot_sensors(ch_type='eeg')
```

Applies the standard biosemi32 montage to each recording and plots the sensor layout.

Figure A.8: PSD before cleaning

```
for session in sessions.iteruples():
    print(f'Computing PSD for participant {session.participant_id} ({session.session_type})')
    raw = session.eeg_raw.copy()
    plt.figure(figsize=(10, 6))
    spectrum = raw.compute_psd(method='welch', fmin=1, fmax=40, verbose=False)
    psd = spectrum.get_data()
    spectrum.plot(amplitude=False)
    plt.title(f'PSD for {session.participant_id}')
    plt.legend()
    plt.grid(True)
    plt.show()
```

Computes and plots the Welch power spectral density (1 to 40 Hz) of each raw recording.

Figure A.9: Epoch plotting

```
for session in sessions.iteruples():
    print(f'Participant: {session.participant_id}')
    raw = session.eeg_raw.copy()
    montage = mne.channels.make_standard_montage('biosemi32')
    raw.drop_channels([ch for ch in raw.ch_names
                      if ch not in montage.ch_names])

    # use the standard montage
    raw.set_montage(montage, verbose=False)
    raw.filter(l_freq=1, h_freq=40, verbose=False)
    epochs = mne.make_fixed_length_epochs(raw, duration=3, preload=True, verbose=False)
    epochs.on_missing = 'warn'
    # plot the data
    epochs.average().detrend().plot_joint()
```

Filters each recording (1 to 40 Hz), cuts it into fixed 3-second epochs and plots the average.

Figure A.10: AutoReject preview

```

for session in sessions.iteruples():
    print(f'Participant: {session.participant_id}')
    raw = session.eeg_raw.copy()
    epochs = mne.make_fixed_length_epochs(raw, duration=3, preload=True, verbose=False)
    epochs.on_missing = 'warn'
    # create an auto-rejection object
    ar = AutoReject(n_interpolate=[1, 2, 3, 4], random_state=11, n_jobs=1, verbose=False)
    # fit on a few epochs to save time
    ar.fit(epochs[:20])
    epochs_ar, reject_log = ar.transform(epochs, return_log=True)
    if len(epochs[reject_log.bad_epochs]) == 0:
        print(f'Dropped epochs for {session.participant_id} is empty')
    else:
        # visualize the dropped epochs
        epochs[reject_log.bad_epochs].plot(scalings=dict(eeg=100e-6), show_scrollbars=False, show_scalesbars=False)
        # visualize the reject log
        reject_log.plot('horizontal')

```

Fits AutoReject on the first 20 epochs of each recording and visualises the rejected epochs and reject log.

Figure A.11: AutoReject with filtering

```

for session in sessions.iteruples():
    print(f'Participant: {session.participant_id}')
    raw = session.eeg_raw.copy()
    raw.filter(1, freq=1, h_freq=30, verbose=False)
    epochs = mne.make_fixed_length_epochs(raw, duration=3, preload=True, verbose=False)
    epochs.on_missing = 'warn'
    # fit on a few epochs to save time
    ar = AutoReject(n_interpolate=[1, 2, 3, 4], random_state=11, n_jobs=1, verbose=False)
    ar.fit(epochs[:20])
    epochs_ar, reject_log = ar.transform(epochs, return_log=True)
    if len(epochs[reject_log.bad_epochs]) == 0:
        print(f'Dropped epochs for {session.participant_id} is empty')
    else:
        # visualize the dropped epochs
        epochs[reject_log.bad_epochs].plot(scalings=dict(eeg=100e-6), show_scrollbars=False, show_scalesbars=False)
        # visualize the reject log
        reject_log.plot('horizontal')

```

Repeats the AutoReject pass on recordings filtered to 1 to 30 Hz.

Figure A.12: ICA artifact removal

```

for session in sessions.iteruples():
    print(f'Participant: {session.participant_id}')
    raw = session.eeg_raw.copy()
    montage = mne.channels.make_standard_montage('biosemi32')
    raw.drop_channels([ch for ch in raw.ch_names
                      if ch not in montage.ch_names])
    # use the standard montage
    raw.set_montage(montage)
    epochs = mne.make_fixed_length_epochs(raw, duration=3, preload=True, verbose=False)
    epochs.on_missing = 'warn'
    # create an auto-rejection object
    ar = AutoReject(n_interpolate=[1, 2, 3, 4], random_state=11, n_jobs=1, verbose=False)
    # fit on a few epochs to save time
    ar.fit(epochs[:20])
    epochs_ar, reject_log = ar.transform(epochs, return_log=True)
    # compute ICA
    ica = mne.preprocessing.ICA(random_state=44)
    print(f'len(epochs): {len(epochs)}\nlen(reject_log.bad_epochs): {len(reject_log.bad_epochs)}')
    ica.fit(epochs[reject_log.bad_epochs])
    # 0: blinks, 2: saccades
    exclude = [0, 2]
    ica.plot_components(exclude)
    ica.exclude = exclude
    ica.plot_overlay(epochs.average(), exclude=ica.exclude)
    ica.apply(epochs, exclude=ica.exclude)

```

Runs ICA on the AutoReject-cleaned epochs and removes the blink and saccade components.

Figure A.13: Cleaning pipeline

```

# create epochs for all EEG in the 'sessions' dataframe
session_cleaned_epochs = []

for session in sessions.iteruples():
    print(f'Participant: {session.participant_id}')
    raw = session.eeg_raw.copy()
    montage = mne.channels.make_standard_montage('biosemi32')
    # drop bad channels (i.e. channels that aren't in the montage's list of channel names)
    raw.drop_channels([ch for ch in raw.ch_names
                      if ch not in montage.ch_names])
    raw.set_montage(montage)
    # high-pass filter the data to remove slow drifts and DC offset
    raw.filter(1, freq=1, h_freq=30, verbose=False)
    # modify the info
    # raw.info = mne.ch_names=ch_names, sfreq=512, ch_types=ch_types)
    epochs = mne.make_fixed_length_epochs(raw, duration=3, preload=True, verbose=False)
    epochs.on_missing = 'warn'
    epochs_num = len(epochs)
    # compute channel-level rejections
    ar = AutoReject(n_interpolate=[1, 2, 3, 4], random_state=11, n_jobs=1, verbose=False)
    ar.fit(epochs[:20]) # fit on the first 20 epochs to save time
    epochs_ar, reject_log = ar.transform(epochs, return_log=True)
    if len(epochs[reject_log.bad_epochs]) == 0:
        print(f'Dropped epochs for {session.participant_id} is empty')
    else:
        epochs[reject_log.bad_epochs].plot(scalings=dict(eeg=100e-6))
        # visualize the reject log
        reject_log.plot('horizontal')
        if len(epochs[reject_log.bad_epochs]) > 0:
            # visualize the cleaned average data
            evoked_bad = epochs[reject_log.bad_epochs].average()
            plt.figure()
            plt.plot(evoked_bad.times, evoked_bad.data.T * 1e6, 'r', zorder=-1)
            epochs_ar.average().plot(axes=plt.gca())
        # inspecting the reject log
        reject_log = ar.get_reject_log(epochs)
        bad_epochs = reject_log.bad_epochs.copy()
        # reject_log.bad_epochs[:] = False # no bad epochs
        # apply the modified reject log to the data
        epochs_ar = ar.transform(epochs, reject_log=reject_log)
        print(f'Number of epochs originally: {epochs_num}, '
              f'after auto-reject: {len(epochs_ar)}')
        session_cleaned_epochs.append(epochs_ar)

```

Applies the full cleaning sequence (channel drop, montage, filter, epoching, AutoReject) to every session and stores the cleaned epochs.

Figure A.14: PSD after cleaning

```

for (session, epochs) in zip(sessions.iteruples(), session_cleaned_epochs):
    print(f'Computing PSD for participant {session.participant_id} ({session.session_type})')
    psd_region = {}
    plt.figure(figsize=(10, 6))
    spectrum = epochs.compute_psd(method='welch', fmin=1, fmax=40, verbose=False)
    psd = spectrum.get_data()
    spectrum.plot(amplitude=False)
    plt.title(f'PSD for {session.participant_id}')
    plt.legend()
    plt.grid(True)
    plt.show()

```

Computes and plots the Welch power spectral density of the cleaned epochs.

Figure A.15: Epoch reshaping

```

eeg_epochs_data = []
eeg_scalogram_column_size = 512

for (session, epochs) in zip(sessions.iteruples(), session_cleaned_epochs):
    epoch_data = epochs.get_data()
    epoch_data_id = epoch_data.flatten()
    # reshape the 2D array into a 2D array with shape (-1, eeg_scalogram_column_size), which means that the number of rows
    # will be inferred based on the total number of elements and the specified number of columns (eeg_scalogram_column_size)
    reshaped_epoch_data = np.reshape(epoch_data_id, (-1, eeg_scalogram_column_size))
    reshaped_epoch_data = reshaped_epoch_data[:min(len(reshaped_epoch_data), 50)]
    print(f'Session: {session.participant_id}, Epochs Reshaped Count: {len(reshaped_epoch_data)}, Status: {session.status}')
    # add the reshaped epoch data to the eeg_epochs_data list along with the session status
    eeg_epochs_data.extend([(lambda x: (x, session.status), reshaped_epoch_data)])
    # eeg_epochs_data.append((epoch_data, session.status))

print(f'Total number of augmented epochs: {len(eeg_epochs_data)}')

```

Flattens the cleaned epochs and reshapes them into rows of 512 samples, keeping at most 50 rows per session.

Figure A.16: Scalogram generation

```

methods = ({'name': 'gaus2'}, {'name': 'cgau7'}, {'name': 'morl'}, )
scales = np.arange(1, 33)

for sample_no, (epoch, status) in enumerate(teeq_epochs_data, start=1):
    for method in methods:
        file_path = f'{data_folder_root}/ds002778_scalograms/{method['name']}/sample_{sample_no}.png'
        method['data_x'] = [] if not 'data_x' in method else method['data_x']
        method['data_y'] = [] if not 'data_y' in method else method['data_y']
        # save time by skipping this routine if the file already exists
        if os.path.exists(file_path):
            method['data_x'] += cv2.imread(file_path)
            method['data_y'].append(status)
            continue
        os.makedirs(os.path.dirname(file_path), exist_ok=True)
        epoch_data = epoch
        # compute the time frequency using the specified method
        coef, freqs = pywt.cwt(epoch_data, scales, method['name'])
        plt.figure(figsize=(10, 5))
        # contourf, img, scalogram, column_size, 33, 1] sets the x-axis from 0 to 'img_scalogram_column_size' and
        # the y-axis from scales[0] to scales[-1] (inverted).
        # The x-axis corresponds to the scales used in the CWT, and the y-axis corresponds to the time points in the EEG epoch.
        plt.imshow(coef[0], extent=[0, img_scalogram_column_size, scales[0], scales[-1]], interpolation='nearest', cmap='plasma',
                  aspect='auto', vmin=abs(coef).max(), vmax=abs(coef).min())
        plt.grid(True)
        plt.xticks(ticks=None, labels=None, minor=False)
        plt.yticks(ticks=None, labels=None, minor=False)
        plt.xticks(ticks=None, labels=None, minor=False)
        plt.tick_params(left=False, right=False, labelleft=False, labelbottom=False, bottom=False)
        plt.savefig(file_path, bbox_inches='tight', pad_inches=0.1)
        plt.close()
        # Clean up memory after saving the image
        gc.collect()
        method['data_x'] += cv2.imread(file_path)
        method['data_y'].append(status)

```

Converts each reshaped epoch to a scalogram image with the gaus2, cgau7, and morl continuous wavelet transforms and saves it as a PNG.

Figure A.17: Scalogram loading

```

methods = ({'name': 'gaus2'}, {'name': 'cgau7'}, {'name': 'morl'}, )
scales = np.arange(1, 33)
method_names = [method['name'] for method in methods]
scalogram_root = f'{data_folder_root}/ds002778_scalograms'

for method in methods:
    paths, labels = [], []
    for status_dir in sorted(os.listdir(scalogram_root)):
        method_dir = f'{scalogram_root}/{status_dir}/{method['name']}'
        if not os.path.isdir(method_dir):
            continue
        for file in sorted(os.listdir(method_dir)):
            if file.endswith('.png'):
                paths.append(f'{method_dir}/{file}')
                labels.append(status_dir)

height, width, channels = cv2.imread(paths[0]).shape
data_x = np.empty((len(paths), height, width, channels), dtype='uint8')
for i, path in enumerate(paths):
    data_x[i] = cv2.imread(path)

method['data_x'] = data_x
method['data_y'] = labels
gc.collect()

```

Loads the saved scalogram PNGs for each method into a single preallocated uint8 array with matching status labels.

Figure A.18: Label encoding

```

# create the categories for the labels (i.e. statuses) based on
# the participant's status (healthy or unhealthy)
categories = dict(zip(sorted(statuses), range(len(statuses))))

for method in methods:
    # one-hot encoding of the statuses
    if isinstance(method['data_y'], list):
        method['data_y'] = to_categorical(np.array([list(map(lambda x: categories[x], method['data_y']))]))
    gc.collect()

# retrieve the number of classes in the labels
num_classes = methods[0]['data_y'].shape[1]
# print the categories dict
print(f'Categories: {categories}')
# print the shape of the images and labels NumPy arrays
print(f'Images shape (sample size, height, width, channels): {methods[0]['data_x'].shape}')
print(f'Labels shape (sample size): {methods[0]['data_y'].shape}')
print(f'Number of classes: {num_classes}')

```

One-hot encodes the healthy and unhealthy status labels for each method.

Figure A.19: Data splitting

```

# split the data into training and testing sets with a ratio of
# 80% training and 20% testing, stratified by the labels
for method in methods:
    gc.collect()
    x_train, x_test, y_train, y_test = train_test_split(
        method['data_x'], method['data_y'], test_size=0.2, stratify=method['data_y'], random_state=42
    )
    # split a validation set from the training set for early stopping,
    # keeping the test set unseen until the final evaluation
    x_train, x_val, y_train, y_val = train_test_split(
        x_train, y_train, test_size=0.15, stratify=y_train, random_state=7
    )
    method['x_train'] = x_train
    method['x_val'] = x_val
    method['y_train'] = y_train
    method['y_val'] = y_val
    method['y_test'] = y_test

```

Splits each method's data into stratified 80% training and 20% testing sets, then carves a validation set from the training set for early stopping.

Figure A.20: Model definition

```

for method in methods:
    print(f'Method: {method['name']}')
    input_shape = method['data_x'][0].shape
    method['model'] = models.Sequential([
        layers.Input(shape=input_shape),
        layers.Rescaling(1. / 255),
        layers.Conv2D(filters=32, kernel_size=(3, 3), activation="relu"),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Conv2D(filters=64, kernel_size=(3, 3), activation="relu"),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Conv2D(filters=64, kernel_size=(3, 3), activation="relu"),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Flatten(),
        layers.Dense(64, activation="relu"),
        layers.Dropout(0.2),
        layers.Dense(len(statuses), activation="softmax")
    ])
    method['model'].compile(
        optimizer="adam",
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )
    method['model'].summary()

```

Builds and compiles the CNN (three 3x3 convolution blocks and a dense head) for each method.

Figure A.21: Model training

```

callback = EarlyStopping(monitor='val_loss', patience=8, restore_best_weights=True)

for method in methods:
    np.random.seed(11)
    tf.random.set_seed(11)
    method['history'] = method['model'].fit(
        method['x_train'],
        method['y_train'],
        epochs=150,
        batch_size=32,
        validation_data=(method['x_val'], method['y_val']),
        callbacks=[callback])

    # print training history timeline for accuracy
    print(f'History metrics {method['name']}: {method['history'].history.keys()}')
    plt.plot(method['history'].history['accuracy'], label='accuracy')
    plt.plot(method['history'].history['val_accuracy'], label='val_accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.legend(['train', 'validation'], loc='upper right')
    plt.show()

    # print training history timeline for loss
    plt.plot(method['history'].history['loss'], label='loss')
    plt.plot(method['history'].history['val_loss'], label='val_loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend(['train', 'validation'], loc='upper right')
    plt.show()

```

Trains each method's model with Adam for up to 150 epochs, stopping early on validation loss and restoring the best weights.

Figure A.22: Test evaluation

```
# generate a no skill predictions list (majority class)
pred_no_skill = np.full((len(methods[0]['y_test']), len(statuses)), 1/len(statuses))

for method in methods:
    test_loss, test_acc = method['model'].evaluate(method['x_test'], method['y_test'])
    print(f"Test accuracy ({method['name']}): {test_acc}")
    method['test_loss'] = test_loss
    method['test_acc'] = test_acc
    # predict probabilities using the model
    method['model_predictions'] = method['model'].predict(method['x_test'])
```

Evaluates each trained model on its test set and builds the no-skill baseline predictions.

Figure A.23: ROC curves

```
for method in methods:
    # calculate roc curves
    for i in range(len(statuses)):
        fpr_no_skill, tpr_no_skill, _ = roc_curve(method['y_test'][i:, 1], pred_no_skill[i:, 1])
        fpr_base_model, tpr_base_model, _ = roc_curve(method['y_test'][i:, 1], method['model_predictions'][i:, 1])
        # plot the roc curve for the no skill, base model, and tuned model
        plt.plot(fpr_no_skill, tpr_no_skill, linestyle='--', label=f'No Skill ({statuses[i]})')
        plt.plot(fpr_base_model, tpr_base_model, marker='.', label=f'Base ({statuses[i]})')

plt.title(f"ROC Curves ({method['name']})")
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.show()
```

Plots the ROC curve of each model against the no-skill baseline.

Figure A.24: AUC scores

```
for method in methods:
    # calculate scores
    auc_no_skill = roc_auc_score(method['y_test'], pred_no_skill)
    auc_base_model = roc_auc_score(method['y_test'], method['model_predictions'])
    print(f"--- ROC AUC Scores with ({method['name']}) method ---")
    # print scores
    print(f'No Skill: {auc_no_skill:.3f}')
    print(f'Base Model: {auc_base_model:.3f}\n')
```

Computes the ROC AUC score of each model and the no-skill baseline.

Figure A.25: Classification metrics

```
# print the confusion matrix, accuracy, precision, specificity, recall, f1 score,
# and classification report given prediction probabilities for each method
for method in methods:
    print(f"--- Metrics from model trained with ({method['name']}) ---")
    predictions_binary = (method['model_predictions'] > 0.5).astype(int)

    # compute the confusion matrix
    cm = confusion_matrix(np.argmax(method['y_test'], axis=1), np.argmax(predictions_binary, axis=1))
    true_negative, false_negative, false_positive, true_positive = cm.ravel()

    # compute performance metrics
    accuracy = (true_positive + true_negative) / (true_positive + true_negative + false_positive + false_negative)
    precision = true_positive / (true_positive + false_positive)
    specificity = true_negative / (true_negative + false_positive)
    recall = true_positive / (true_positive + false_negative)
    f1_score = 2 * (precision * recall) / (precision + recall)

    # plot the confusion matrix
    plt.figure(figsize=(6, 6))
    plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
    plt.title(f"Confusion Matrix ({method['name']})")
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.colorbar()

    # print the values obtained
    print(f"Confusion Matrix:")
    print(cm)
    print(f"Accuracy: {accuracy}")
    print(f"Precision: {precision}")
    print(f"Specificity: {specificity}")
    print(f"Recall: {recall}")
    print(f"F1 Score: {f1_score}")

    # print a class-based report
    print(f"Classification Report:")
    print(classification_report(np.argmax(method['y_test'], axis=1), np.argmax(method['model_predictions'], axis=1)))
```

Prints the confusion matrix, accuracy, precision, specificity, recall, F1 score and classification report of each model.

Figure A.26: Cross-validation

```
def build_model(input_shape):
    model = models.Sequential([
        layers.Input(shape=input_shape),
        layers.Flatten(), # 253,
        layers.Conv2D(filters=32, kernel_size=(3, 3), activation='relu'),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Conv2D(filters=64, kernel_size=(3, 3), activation='relu'),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Conv2D(filters=64, kernel_size=(3, 3), activation='relu'),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Flatten(),
        layers.Dense(64, activation='relu'),
        layers.Dropout(0.2),
        layers.Dense(len(statuses), activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    return model

for method in methods:
    kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=44)
    cv_scores = []

    for train, val in kfold.split(method['data_x'], method['data_y'].argmax(axis=1)):
        # release the previous fold's model before building the next one
        if keras.backend.clear_session():
            gc.collect()
        # split an inner validation set from the training folds for early stopping,
        # so the fold's held-out data stays unseen until evaluation
        train_idx, inner_val_idx = train_test_split(
            train, test_size=0.15, stratify=method['data_y'][train].argmax(axis=1), random_state=7)
        # a fresh model per fold, so no fold sees weights trained on another fold
        fold_model = build_model(method['data_x'][train_idx].shape)
        fold_model.fit(
            method['data_x'][train_idx],
            method['data_y'][train_idx],
            validation_data=(method['data_x'][inner_val_idx], method['data_y'][inner_val_idx]),
            epochs=150,
            batch_size=32,
            callbacks=[EarlyStopping(monitor='val_loss', patience=8, restore_best_weights=True)],
            verbose=0)
        scores = fold_model.evaluate(method['data_x'][val], method['data_y'][val])
        # print and track the validation accuracy for this fold or round
        print(f"Validation accuracy ({method['name']}) : {scores[1]:.2f}%")
        cv_scores.append(scores[1] * 100)

    print(f"All accuracy ({method['name']}) : {cv_scores}")
    print(f"Mean validation accuracy ({method['name']}) : {np.mean(cv_scores):.2f}% (+ {np.std(cv_scores):.2f}%)"
```

Runs stratified five-fold cross-validation, training a fresh model per fold with early stopping on an inner validation split.

ACKNOWLEDGEMENT

I would like to thank my supervisor Mrs. Divya Stinamaluwa for her guidance and support throughout this research.

REFERENCES

- [1] S. A. Chaudhary, S. Chaudhary, and S. Rawat, "Understanding Parkinson's disease: Current trends and its multifaceted complications," Front. Aging Neurosci., vol. 17, Art. no. 1617106, 2025, doi: 10.3389/fnagi.2025.1617106.
- [2] C. E. Clarke, "Parkinson's disease," BMJ, vol. 335, no. 7617, pp. 441–445, Sep. 2007.
- [3] A. Lemsky, "Oscillatory signatures of Parkinson's disease: Central and peripheral EEG correlates," Mar. 2025, doi: 10.48550/arXiv.2503.12392.
- [4] A. J. Hughes, S. E. Daniel, Y. Ben-Shlomo, and A. J. Lees, "The accuracy of diagnosis of parkinsonian syndromes in a specialist movement disorder service," Brain, vol. 125, no. 4, pp. 861–870, Apr. 2002, doi: 10.1093/brain/awf080.
- [5] G. Rizzo, M. Copetti, S. Arcuti, D. Martino, A. Fontana, and G. Logroscino, "Accuracy of clinical

- diagnosis of Parkinson disease: A systematic review and meta-analysis," *Neurology*, vol. 86, no. 6, pp. 566–576, Feb. 2016, doi: 10.1212/WNL.0000000000002350.
- [6] A. Azadnouran, H. Akbari, M. T. Sadiq, D. Smith, and M. Mete, "Comparative evaluation of time–frequency transformations and pretrained CNN models for EEG-based Parkinson's disease detection," *BioMedInformatics*, vol. 6, no. 2, Art. no. 12, 2026, doi: 10.3390/biomedinformatics6020012.
- [7] M. A. Motin, M. Mahmud, and D. J. Brown, "Detecting Parkinson's disease from electroencephalogram signals: An explainable machine learning approach," in *Proc. 2022 IEEE 16th Int. Conf. Appl. Inf. Commun. Technol. (AICT)*, Washington, DC, USA, 2022, pp. 1–6, doi: 10.1109/AICT55583.2022.10013589.
- [8] A. P. Rockhill, N. Jackson, J. George, A. Aron, and N. C. Swann, "UC San Diego resting state EEG data from patients with Parkinson's disease," *OpenNeuro*, 2021. [Dataset]. doi: 10.18112/openneuro.ds002778.v1.0.5.
- [9] Y. Luo, L. Qiao, M. Li, X. Wen, W. Zhang, and X. Li, "Global, regional, national epidemiology and trends of Parkinson's disease from 1990 to 2021: Findings from the Global Burden of Disease Study 2021," *Front. Aging Neurosci.*, vol. 16, Art. no. 1498756, 2024, doi: 10.3389/fnagi.2024.1498756.
- [10] Parkinson's Foundation, "Parkinson's disease statistics: Prevalence, incidence, and global trends." Accessed: Aug. 24, 2026. [Online]. Available: <https://www.parkinson.org/understanding-parkinsons/statistics>.
- [11] H. Khachnaoui, N. Khelifa, and R. Mabrouk, "Machine learning for early Parkinson's disease identification within SWEDD group using clinical and DaTSCAN SPECT imaging features," *J. Imaging*, vol. 8, no. 4, Art. no. 97, 2022, doi: 10.3390/jimaging8040097.
- [12] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998, doi: 10.1109/5.726791.
- [13] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, Jun. 2017, doi: 10.1145/3065386.
- [14] B. Sistaninejad, H. Rasi, and P. Nayeri, "A review paper about deep learning for medical image analysis," *Comput. Math. Methods Med.*, vol. 2023, Art. no. 7091301, 2023, doi: 10.1155/2023/7091301.
- [15] S. O'Connell, D. M. Cannon, and P. Ó Broin, "Predictive modelling of brain disorders with magnetic resonance imaging: A systematic review of modelling practices, transparency, and interpretability in the use of convolutional neural networks," *Hum. Brain Mapp.*, vol. 44, no. 18, pp. 6561–6574, Dec. 2023, doi: 10.1002/hbm.26521.
- [16] C. Torrence and G. P. Compo, "A practical guide to wavelet analysis," *Bull. Amer. Meteorol. Soc.*, vol. 79, no. 1, pp. 61–78, Jan. 1998.
- [17] F. Li, F. He, F. Wang, D. Zhang, Y. Xia, and X. Li, "A novel simplified convolutional neural network classification algorithm of motor imagery EEG signals based on deep learning," *Appl. Sci.*, vol. 10, no. 5, Art. no. 1605, 2020, doi: 10.3390/app10051605.
- [18] T. Wang, C. Lu, Y. Sun, M. Yang, C. Liu, and C. Ou, "Automatic ECG classification using continuous wavelet transform and convolutional neural network," *Entropy*, vol. 23, no. 1, Art. no. 119, 2021, doi: 10.3390/e23010119.
- [19] A. I. Korda, E. Ventouras, P. Asvestas, M. Toumaian, G. K. Matsopoulos, and N. Smyrnis, "Convolutional neural network propagation on electroencephalographic scalograms for detection of schizophrenia," *Clin. Neurophysiol.*, vol. 139, pp. 90–105, Jul. 2022, doi: 10.1016/j.clinph.2022.04.010.
- [20] S. Bagherzadeh, K. Maghooli, A. Shalbaf, and A. Maghsoudi, "A hybrid EEG-based emotion recognition approach using wavelet convolutional neural networks and support vector machine," *Basic Clin. Neurosci.*, vol. 14, no. 1, pp. 87–102, 2023, doi: 10.32598/bcn.2021.3133.1.
- [21] D. Goyal and B. S. Pabla, "Condition based maintenance of machine tools—A review," *CIRP J. Manuf. Sci. Technol.*, vol. 10, pp. 24–35, Jul. 2015, doi: 10.1016/j.cirpj.2015.05.004.

[22] S. G. Mallat, "A theory for multiresolution signal decomposition: The wavelet representation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 11, no. 7, pp. 674–693, Jul. 1989, doi: 10.1109/34.192463.

[23] D. Valero-Carreras, J. Alcaraz, and M. Landete, "Comparing two SVM models through different metrics based on the confusion matrix," *Comput. Oper. Res.*, vol. 152, Apr. 2023, Art. no. 106131, doi: 10.1016/j.cor.2022.106131.

[24] Ş. K. Çorbacıoğlu and G. Aksel, "Receiver operating characteristic curve analysis in diagnostic accuracy studies: A guide to interpreting the area under the curve value," *Turkish J. Emerg. Med.*, vol. 23, no. 4, pp. 195–198, Oct. 2023, doi: 10.4103/tjem.tjem_182_23.

[25] A. de Hond, E. Steyerberg, and B. van Calster, "Interpreting area under the receiver operating characteristic curve," *Lancet Digit. Health*, vol. 4, no. 12, pp. e853–e855, Dec. 2022, doi: 10.1016/S2589-7500(22)00188-1.

[26] S. Bates, T. Hastie, and R. Tibshirani, "Cross-Validation: What Does It Estimate and How Well Does It Do It?" *Journal of the American Statistical Association*, vol. 119, no. 546, pp. 1434–1445, May, 2023, doi: 10.1080/01621459.2023.2197686.

[27] Alexandre Gramfort, Martin Luessi, Eric Larson, Denis A. Engemann, Daniel Strohmeier, Christian Brodbeck, Roman Goj, Mainak Jas, Teon Brooks, Lauri Parkkonen, and Matti S. Hämäläinen, "MEG and EEG data analysis with MNE-Python," *Frontiers in Neuroscience*, vol. 7, no. 267, pp. 1–13, 2013, doi:10.3389/fnins.2013.00267.